Practical No. 10

Name: Vaishali Kalapad

Class : SY-A

Branch – AI & DS

Roll no : A-49

Aim - Version Control using Git & GitHub Questions

:

1. **What is the difference between Git and GitHub?**

Ans : Git vs GitHub Git:

• Git is a Version Control System (VCS)

• Created by Linus Torvalds in 2005

• Works completely offline on your local machine

• Tracks every change made to your code

• Stores full project history locally

• Free and open-source

• Used via command line (terminal)

• Alternatives: Mercurial, SVN GitHub:

• GitHub is a cloud-based hosting platform for Git repos

• Founded in 2008, bought by Microsoft in 2018

• Requires internet connection

• Adds collaboration features: Pull Requests, Issues, Actions

• Stores your code remotely on servers

• Has free and paid plans

• Alternatives: GitLab, Bitbucket

2. Explain the difference between:

- **git add**
- **git commit Ans :** git add:

• Moves files from Working Directory → Staging Area

• Tells Git which changes to include in next commit

• Does NOT save permanently

• Can stage one file, multiple files, or all files

• Reversible using git restore –staged

Commands:

Git add file.txt → one file

Git add . → all files

Git add *.js → all JS files

Git add -p → select parts interactively Git

commit:

• Moves files from Staging Area → Local Repository

• Creates a permanent snapshot in history

• Each commit gets a unique SHA-1 hash ID

• Requires a commit message • Records author, timestamp, and changes Commands:

Git commit -m "message" → basic commit

Git commit –amend → edit last commit Git commit -

am "message" → add + commit together

3.What is the staging area in Git?

Ans : • Also called the Index

• It is a middle layer between working directory and repository

• Physically stored in .git/index file

• Lets you selectively choose what to commit

• You can modify 10 files but commit only 2

• Helps create clean, logical commits • You can review staged changes before saving

- Commands:

Git status → see staged/unstaged files

Git diff –staged → see staged changes

Git restore –staged file → unstage a file

- 4 States of a file:

• Untracked → new file Git doesn't know about

• Modified → tracked file with new changes

• Staged → ready to be committed

• Committed → permanently saved in history

4**. What happens if you modify a file after committing it?**

Ans : • The old committed version stays safe in history

• Modified file appears as "modified" in working directory

• Git detects change by comparing file hash with last commit

• Git status shows it under "Changes not staged for commit"

• You must add and commit again to save new version

• Each commit gets a new unique hash

• Old commit is still accessible anytime

• You can go back using git checkout <old-hash>

• Nothing in Git is truly lost after committing • To fix last commit quickly: git commit –amend Process after modifying:

Modify file → git add → git commit → new snapshot created

5.Difference between:

- git push

- git pull

Ans : git push:

• Uploads local commits → remote repository (GitHub)

• Shares your work with teammates

• Only pushes committed changes (not just staged)

• May be rejected if remote has newer commitsCommands:

- Git push origin main → push to main branch
- Git push origin branch-name → push specific branch
- Git push -u origin main → set default upstream
- Git push –force → force overwrite (dangerous) Git

pull:

• Downloads remote changes → local machine

• Keeps your local repo up to date

• Combines: git fetch + git merge

• Can cause merge conflicts if same lines were

editedCommands:

- Git pull origin main → pull from main

-Git pull –rebase → pull and rebase instead of merge

6 .What is Version Control? Differentiate between Centralized v/s Distributed VCS.

Ans : Version Control:

• System that records changes to files over time

• Tracks who, what, when, and why something changed

• Allows reverting to any previous version

• Enables parallel work without overwriting each other

• Essential for team software development Centralized VCS

(CVCS):

• One central server holds the entire repository

• Developers check out only one version (not full history)

• All operations need server/internet connection

• Examples: SVN, CVS, Perforce - Advantages:

• Simple to understand

• Admin has full control

• Good for binary/design files Distributed VCS (DVCS):

• Every developer has full copy of entire repo including history

• Most operations happen locally (fast)

• Remote server used only for syncing/sharing

• Examples: Git, Mercurial

Advantages:- • Fully

offline capability

• Every clone = complete backup

• Very fast operations

• Easy branching and merging• No single point of failure**Theroy**

:

```
ketan@Zoro MINGW64 ~ (main)
$ git config --global user.name "Vaibhav"

ketan@Zoro MINGW64 ~ (main)
$ git config --global user.email "vaibhavjarhad14@gmail.com"

ketan@Zoro MINGW64 ~ (main)
$ git config --global --list
user.name=Vaibhav
user.enail=ketankale2426d@gmail.com
user.email=vaibhavjarhad14@gmail.com

ketan@Zoro MINGW64 ~ (main)
$ mkdir MyProjectt

ketan@Zoro MINGW64 ~ (main)
$ cd MyProjectt

ketan@Zoro MINGW64 ~/MyProjectt (main)
$ git init
Initialized empty Git repository in C:/Users/ketan/MyProjectt/.git/

ketan@Zoro MINGW64 ~/MyProjectt (master)
$ git log
```

```
ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git commit -m "First commit"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ echo "Helli Git" > hello.txt

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git commit -m "First commit"
[master (root-commit) 68f66c4] First commit
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$
```

```
MINGW64:/c/Users/ketan/MyProjecttt

ketan@Zoro MINGW64 ~ (main)
$ git config --global --list
user.name=Vaibhav
user.enail=ketankale2426d@gmail.com
user.email=vaibhavjarhad14@gmail.com

ketan@Zoro MINGW64 ~ (main)
$ mkdir MyProjecttt

ketan@Zoro MINGW64 ~ (main)
$ cd MyProjecttt

ketan@Zoro MINGW64 ~/MyProjecttt (main)
$ git init
Initialized empty Git repository in C:/Users/ketan/MyProjecttt/.git/

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git init
Reinitialized existing Git repository in C:/Users/ketan/MyProjecttt/.git/

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$
```

```
ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git log
commit 68f66c4d313d4312de19418150545fbf4ecf7d3c (HEAD -> master)
Author: Vaibhav <vaibhavjarhad14@gmail.com>
Date:   Fri May 8 13:38:00 2026 +0530

    First commit

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git remote add origin
usage: git remote add [<options>] <name> <url>

    -f, --[no-]fetch      fetch the remote branches
    --[no-]tags           import all tags and associated objects when fetching
                          or do not fetch any tag at all (--no-tags)
    -t, --[no-]track <branch>
                          branch(es) to track
    -m, --[no-]master <branch>
                          master branch
    --[no-]mirror[=(push|fetch)]
                          set up remote as a mirror to push to or fetch from


ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git remote add origin https://github.com/vaibhavjarhad14-cmd/MyProjecttt.git

ketan@Zoro MINGW64 ~/MyProjecttt (master)
$ git push  -u origin master
```

```
MINGW64:/c/Users/ketan

ketan@Zoro MINGW64 ~ (main)
$ git-- version
bash: git--: command not found

ketan@Zoro MINGW64 ~ (main)
$ git --version
git version 2.54.0.windows.1

ketan@Zoro MINGW64 ~ (main)
$ git config --global user.name "Vaibhav"

ketan@Zoro MINGW64 ~ (main)
$ git config --global user.email "vaibhavjarhad14@gmail.com"

ketan@Zoro MINGW64 ~ (main)
$ git config --global --list
user.name=Vaibhav
user.enail=ketankale2426d@gmail.com
user.email=vaibhavjarhad14@gmail.com

ketan@Zoro MINGW64 ~ (main)
$ |
```

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

1 **General**

Owner *

vaibhavjarhad14-cmd /

Repository name *

MyProjecttt

MyProjecttt is available.

Great repository names are short and memorable. How about reimagined-system?

Description

0 / 350 characters